CLOUD COMPUTING | PRACTICAL SESSION

Lab 4

Deploying a Multi-Tier Application with Docker Compose

Lab guide and take-home assignment (Ubuntu 24.04, Docker Engine, Docker Compose, Nginx, Python Flask, MySQL)

Item	Details
Course	
Instructor	
Student name	
Student ID / Group	
Lab date	
Lab time	About 2.5 hours
Assignment due	One week after the lab (Part 2)

Lab at a glance

Topic	Details
Goal	Deploy a multi-tier application as **three containers** (frontend, backend, database) and understand how the layers communicate, persist data and are monitored.
Application	**Task Manager**: a tiny app to view, add and delete tasks. All code is given; you focus on the deployment.
What you need	A laptop running Ubuntu 24.04 with internet access and `sudo` rights. No cloud account is required.
Important limit	All containers run on **one machine**. This is a multi-container application, **not** a multi-node cluster. In a cloud deployment, each tier would run on its own virtual machine.

PART 1 - LAB

1. Objectives

After this lab you will be able to:

1. Install and verify Docker Engine and the Docker Compose plugin.
2. Download images from Docker Hub and build your own image from a Dockerfile.
3. Deploy a frontend, backend and database as separate containers using one `compose.yaml` file.
4. Explain how the tiers communicate through Docker networks and port mapping.
5. Use a Docker volume so that database data survives container replacement.
6. Inspect, monitor and troubleshoot containers using status commands and logs.

2. Key terms

Term	Meaning
Image	A packaged, read-only template (for example `mysql:8.4`). Images come from Docker Hub or are built from a Dockerfile.
Container	A running instance of an image. You can delete and recreate it at any time.
Docker Hub	A public registry where images are stored and downloaded.
Dockerfile	A text recipe that describes how to build an image.
Docker Compose	A tool that starts several containers, networks and volumes from one YAML file.
Port mapping	`8080:80` forwards port 8080 on your laptop to port 80 inside the container.
Network	A private virtual network. Containers on the same network reach each other **by service name** (for example `backend`).
Volume	Storage managed by Docker that lives outside the container, so data survives when the container is removed.

3. Architecture

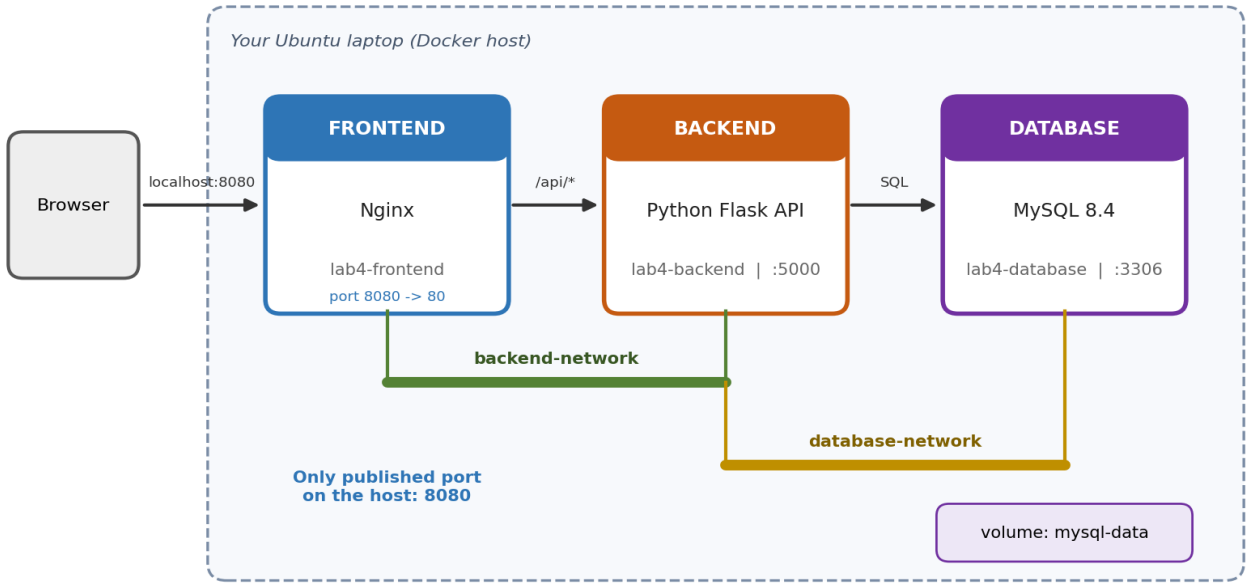


Figure 1: Three containers, two private networks and one volume

Tier	Technology	Responsibility
Frontend	Nginx	Serves the web page and forwards `/api/` requests to the backend
Backend	Python Flask	API (URLs) and business rules; talks to the database
Database	MySQL 8.4	Stores the tasks permanently

Request path when you press **Add Task**: Browser, then Nginx (port 8080 on your laptop), then Flask (port 5000), then MySQL (port 3306).

> **NOTE:** Only the frontend publishes a port to your laptop (8080). The backend and database have **no published port**, so they cannot be reached from outside Docker. The database also sits on its own network (`database-network`), so even the frontend container cannot reach it.

4. Step 1 - Install Docker on Ubuntu

Open a Terminal. Ubuntu runs Docker natively, so WSL2 or Docker Desktop is **not** needed.

4.1 Check whether Docker is already installed

```
docker --version
docker compose version
```

If both commands print a version number, skip to Step 2. Otherwise continue.

4.2 Add Docker's official repository

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Copy the whole block, including the first and last line (EOF)

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

4.3 Install and start Docker

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager
```

The status should show `active (running)`.

4.4 Test Docker

```
sudo docker run hello-world
sudo docker --version
sudo docker compose version
```

CHECKPOINT: `hello-world` prints a success message. Take **Screenshot 1** showing this output and the two version numbers.

NOTE: This lab uses `sudo docker ...` for every command, so no extra setup is needed. (You can avoid `sudo` by adding yourself to the `docker` group, but that gives your user root-level power over the machine, so we do not do it here.)

5. Step 2 - Download images from Docker Hub

Image	Purpose
`nginx:alpine`	Frontend web server (small Alpine Linux version)
`python:3.12-slim`	Base for our backend image (we add our own code)
`mysql:8.4`	Database server

```
sudo docker pull nginx:alpine
sudo docker pull python:3.12-slim
sudo docker pull mysql:8.4
sudo docker images
```

You should see the three images listed. (Compose would download them automatically later; pulling them first lets you see what is being used.)

6. Step 3 - Create the project

```
mkdir -p ~/docker-multitier-lab/backend ~/docker-multitier-lab/frontend
cd ~/docker-multitier-lab
pwd
```

At the end your project will look like this. You will create each file with the `nano` editor (`nano filename`, paste the content, save with `Ctrl+O`, `Enter`, then exit with `Ctrl+X`).

```
docker-multitier-lab/
```

```
|-- compose.yaml
|-- .env
|-- backend/
|   |-- Dockerfile
|   |-- requirements.txt
|   `-- app.py
`-- frontend/
    |-- index.html
    `-- nginx.conf
```

> **WARNING:** Always run the `docker compose` commands from inside the `~/docker-multitier-lab` folder. Compose looks for `compose.yaml` in the current folder.

7. Step 4 - Create the backend tier (API and business logic)

The backend receives requests from the frontend and talks to MySQL. Notice that `app.py` separates the **business logic** (the rules) from the **API** (the URLs).

7.1 Python packages

File: backend/requirements.txt

```
Flask
mysql-connector-python
```

7.2 The application

File: backend/app.py

```
import os
import time

import mysql.connector
from flask import Flask, jsonify, request
from mysql.connector import Error

app = Flask(__name__)

# Connection settings come from environment variables (set in compose.yaml)
DB_HOST = os.environ.get("DB_HOST", "db")
DB_NAME = os.environ["MYSQL_DATABASE"]
DB_USER = os.environ["MYSQL_USER"]
DB_PASSWORD = os.environ["MYSQL_PASSWORD"]


# ---------- Database access ----------
def get_connection():
    return mysql.connector.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        database=DB_NAME,
    )


def initialize_database():
    """Create the table. Retry, because MySQL may still be starting."""
    for attempt in range(30):
        try:
            connection = get_connection()
            cursor = connection.cursor()
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS tasks (
                    id INT AUTO_INCREMENT PRIMARY KEY,
                    title VARCHAR(255) NOT NULL
                )
            """)
            connection.commit()
            cursor.close()
            connection.close()
            print("Database initialized successfully.", flush=True)
            return
        except Error as error:
            print(f"Database not ready ({attempt + 1}/30): {error}", flush=True)
            time.sleep(2)
    raise RuntimeError("Could not connect to the database.")


# ---------- Business logic (the rules of the application) ----------
def clean_title(raw_title):
    """Return the cleaned title, or None if it breaks the rules."""
    title = str(raw_title or "").strip()
    if not title or len(title) > 255:
        return None
    return title


# ---------- API (the URLs the frontend can call) ----------
@app.route("/api/health", methods=["GET"])
def health():
    try:
        get_connection().close()
        return jsonify({"api": "running", "database": "connected"})
    except Error:
        return jsonify({"api": "running", "database": "unavailable"}), 503
```

```
@app.route("/api/tasks", methods=["GET"])
def get_tasks():
    connection = get_connection()
    cursor = connection.cursor(dictionary=True)
    cursor.execute("SELECT id, title FROM tasks ORDER BY id")
    tasks = cursor.fetchall()
    cursor.close()
    connection.close()
    return jsonify(tasks)


@app.route("/api/tasks", methods=["POST"])
def add_task():
    data = request.get_json(silent=True) or {}
    title = clean_title(data.get("title"))
    if title is None:
        return jsonify({"error": "Task title must be 1 to 255 characters"}), 400

    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("INSERT INTO tasks (title) VALUES (%s)", (title,))
    connection.commit()
    task_id = cursor.lastrowid
    cursor.close()
    connection.close()
    return jsonify({"id": task_id, "title": title}), 201


@app.route("/api/tasks/<int:task_id>", methods=["DELETE"])
def delete_task(task_id):
    connection = get_connection()
    cursor = connection.cursor()
    cursor.execute("DELETE FROM tasks WHERE id = %s", (task_id,))
    connection.commit()
    deleted = cursor.rowcount
    cursor.close()
    connection.close()
    if deleted == 0:
        return jsonify({"error": "Task not found"}), 404
    return jsonify({"message": "Task deleted"})


if __name__ == "__main__":
    initialize_database()
    app.run(host="0.0.0.0", port=5000)
```

> **NOTE:** `host="0.0.0.0"` lets Flask accept connections from other containers. If it listened only on `127.0.0.1`, no other container could reach it. The database password is **not** in the code; it comes from environment variables.

7.3 The Dockerfile

File: backend/Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Instruction	Explanation
FROM	Start from the official Python image
WORKDIR	Working folder inside the image
COPY	Copy files from your project into the image
RUN	Run a command while building (installs the packages)
EXPOSE	Documents the port the app uses
CMD	Command that starts the API when the container runs

8. Step 5 - Create the frontend tier

8.1 The web page

File: frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Docker Multi-Tier Lab</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px;
               margin: 40px auto; padding: 0 16px; }
        input { width: 65%; padding: 8px; }
        button { padding: 8px 12px; cursor: pointer; }
        li { margin: 10px 0; }
        #message { color: #b00020; }
    </style>
</head>
<body>
    <h1>Cloud Computing Lab 4</h1>
    <h2>Task Management Application</h2>
    <p>Frontend: Nginx | Backend: Python Flask | Database: MySQL</p>

    <form id="task-form">
        <input id="task-title" type="text" placeholder="Enter a task" required>
        <button type="submit">Add Task</button>
    </form>

    <h3>Tasks</h3>
    <ul id="tasks"></ul>
    <p id="message"></p>

    <script>
        const taskList = document.getElementById("tasks");
        const message = document.getElementById("message");

        async function loadTasks() {
            try {
                const response = await fetch("/api/tasks");
                if (!response.ok) throw new Error("Could not load tasks");

                const tasks = await response.json();
                taskList.replaceChildren();

                tasks.forEach(task => {
                    const item = document.createElement("li");
                    const text = document.createElement("span");
                    const del = document.createElement("button");

                    text.textContent = task.title + " ";
                    del.textContent = "Delete";
                    del.addEventListener("click", async () => {
                        const result = await fetch("/api/tasks/" + task.id, { method: "DELETE" });
                        if (!result.ok) message.textContent = "Could not delete the task";
                        loadTasks();
                    });

                    item.append(text, del);
                    taskList.appendChild(item);
                });
                message.textContent = "";
            } catch (error) {
                message.textContent = error.message;
            }
        }

        document.getElementById("task-form").addEventListener("submit", async event => {
```

```
            event.preventDefault();
            const input = document.getElementById("task-title");
            const title = input.value.trim();
            if (!title) return;

            try {
                const response = await fetch("/api/tasks", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({ title })
                });
                if (!response.ok) throw new Error("Could not add task");
                input.value = "";
                await loadTasks();
            } catch (error) {
                message.textContent = error.message;
            }
        });

        loadTasks();
    </script>
</body>
</html>
```

8.2 The Nginx configuration

File: frontend/nginx.conf

```
server {
    listen 80;

    root /usr/share/nginx/html;
    index index.html;

    # Static files (the web page)
    location / {
        try_files $uri $uri/ =404;
    }

    # Forward API calls to the backend container
    location /api/ {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

How it works: the browser loads `index.html` from Nginx. The page calls `/api/tasks` on its own address; Nginx forwards that to `http://backend:5000`. The name `backend` is the Compose service name, which Docker resolves automatically. The browser never talks to the backend or database directly.

9. Step 6 - Configure everything with Docker Compose

9.1 The settings file (.env)

Compose automatically reads a file named `.env` and substitutes the values into `compose.yaml`. This keeps passwords out of the main configuration file.

File: .env

```
MYSQL_ROOT_PASSWORD=labroot123
MYSQL_DATABASE=tasksdb
MYSQL_USER=taskuser
MYSQL_PASSWORD=taskpass123
```

> **WARNING:** These are demonstration passwords for a local lab. In real projects use strong passwords, never commit `.env` to Git, and never expose a database to the internet.

9.2 The Compose file

File: compose.yaml (YAML indentation matters: use spaces, never tabs)

```
services:

  frontend:
    image: nginx:alpine
    container_name: lab4-frontend
    ports:
      - "8080:80"
    volumes:
      - ./frontend/index.html:/usr/share/nginx/html/index.html:ro
      - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - backend
    networks:
      - backend-network
    restart: unless-stopped

  backend:
    build: ./backend
    image: lab4-backend
    container_name: lab4-backend
    environment:
      DB_HOST: db
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
    networks:
      - backend-network
      - database-network
    restart: unless-stopped

  db:
    image: mysql:8.4
    container_name: lab4-database
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test:
        - CMD-SHELL
        - mysqladmin ping -h localhost -uroot -p"$${MYSQL_ROOT_PASSWORD}" --silent
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
    networks:
      - database-network
    restart: unless-stopped

volumes:
  mysql-data:

networks:
  backend-network:
  database-network:
```

9.3 Understanding the file

Part	What it does
`ports: "8080:80"`	Format `HOST_PORT:CONTAINER_PORT`. Your laptop's port 8080 goes to port 80 in the Nginx container.
`build: ./backend`	Builds the backend image from the Dockerfile. `image: lab4-backend` gives that image a clear name.
`environment`	Passes settings into the container. For MySQL, the database and user are created the **first time** it starts with an empty data folder.
`volumes: mysql-data`	A named volume holds `/var/lib/mysql`, so the data survives when the container is removed.
`networks`	`backend-network` connects frontend and backend. `database-network` connects backend and database. The frontend is **not** on the database network.
`healthcheck` and `depends_on`	The backend starts only when MySQL reports "healthy", which is more reliable than waiting for the container to merely start.
`restart: unless-stopped`	Docker restarts the container if it crashes or after a reboot.

10. Step 7 - Validate the configuration

```
cd ~/docker-multitier-lab
find . -maxdepth 2 -type f
sudo docker compose config --quiet && echo "Compose file is valid"
```

`find` should list the 7 files from the project tree (including `.env`). If `config` shows an error, check the indentation in `compose.yaml`.

11. Step 8 - Build and start the application

```
sudo docker compose up -d --build
```

Docker downloads missing images, builds the backend image, creates the networks and volume, starts MySQL, waits until it is healthy, then starts the backend and the frontend. `-d` runs everything in the background. **The first run can take several minutes.**

Check status and logs (press Ctrl+C to stop following logs; the containers keep running)

```
sudo docker compose ps
sudo docker compose logs backend
sudo docker compose logs db
sudo docker compose logs -f
```

Service	Expected status
frontend	running
backend	running
db	running (healthy)

> **CHECKPOINT:** All three services are up. Take **Screenshot 2**: the output of `sudo docker compose ps`.

12. Step 9 - Test the application

12.1 In the browser

Open **http://localhost:8080**. Add the tasks `Learn Docker Compose` and `Understand container networking`. Delete one of them with its **Delete** button. Try to add a task made only of spaces: nothing is added (business rule).

> **CHECKPOINT:** Take **Screenshot 3**: the application in the browser with the address bar visible.

12.2 With the terminal (the API)

```
curl http://localhost:8080/api/health
curl http://localhost:8080/api/tasks
curl -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" -d '{"title":"Practice Linux commands"}'
curl -i -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" -d '{"title":"   "}'
curl -X DELETE http://localhost:8080/api/tasks/1
```

Command	Expected result
health	`{"api":"running","database":"connected"}`
list tasks	A JSON list of your tasks, for example `[{"id":1,"title":"..."}]`
add task	HTTP 201 and the new task with its id (the id may differ on your machine)
spaces only	**HTTP 400** with an error message: the business rule works
delete	`{"message":"Task deleted"}`, or HTTP 404 if that id does not exist

12.3 Look inside the database

Prove that the data is really stored in MySQL (enter the `MYSQL_PASSWORD` from `.env` when asked):

```
sudo docker compose exec db mysql -utaskuser -p tasksdb -e "SELECT * FROM tasks;"
```

> **CHECKPOINT:** Take **Screenshot 4**: the SQL result showing the same tasks as the browser.

13. Step 10 - Explore the containers

13.1 Containers and images

```
sudo docker ps
sudo docker ps -a
sudo docker images
```

Which images came from Docker Hub? Which one did you build yourself? (Hint: `lab4-backend`.)

13.2 Networks

```
sudo docker network ls
sudo docker network inspect docker-multitier-lab_backend-network
sudo docker network inspect docker-multitier-lab_database-network
```

Find the `Containers` section in each output. Which containers are on which network? (If the names differ, copy them from `docker network ls`.)

13.3 Volume

```
sudo docker volume ls
sudo docker volume inspect docker-multitier-lab_mysql-data
```

13.4 A shell inside a container

```
sudo docker exec -it lab4-backend sh
hostname
cat /etc/os-release
exit
```

13.5 The database is not exposed

Run these three commands

```
curl -s --max-time 3 localhost:3306; echo "exit code: $?"
sudo docker compose exec frontend ping -c 1 backend
sudo docker compose exec frontend ping -c 1 db
```

The first command fails because no port 3306 is published on your laptop (unless you have MySQL installed locally). The frontend can reach `backend` but **not** `db`, because they share no network.

> **CHECKPOINT:** Take **Screenshot 5**: the output of the network inspect commands and the three commands above.

14. Step 11 - Test data persistence

1. In the browser, add a task called `Persistence Test`.
2. Remove the containers: `sudo docker compose down` (this removes the containers and networks, but **keeps the volume**).
3. Check that nothing is running: `sudo docker compose ps`.
4. Start again: `sudo docker compose up -d` and wait until `sudo docker compose ps` shows the services running.
5. Open http://localhost:8080. The task `Persistence Test` is still there.

> **THINK:** The database container was deleted and recreated. Why did the data survive? Which line of `compose.yaml` is responsible?

> **CHECKPOINT:** Take **Screenshot 6**: the page after the restart, showing `Persistence Test`.

15. Step 12 - Experiments: see the dependencies

#	Action	Question	Restore
1	`sudo docker compose stop backend`	Refresh the page. What do you see? Which tier failed? Is the frontend still running?	`sudo docker compose start backend`, then `sudo docker compose restart frontend`
2	`sudo docker compose stop db`	Run `curl -i localhost:8080/api/health`. What is the status code and message?	`sudo docker compose start db` (wait about 30 seconds)
3	`sudo docker kill lab4-backend`, wait 10 seconds, then `sudo docker compose ps`	Does the backend return by itself? Which setting makes this happen?	Automatic

16. Troubleshooting

Golden rule: read the logs first (`sudo docker compose logs --tail=100 backend`), and test one tier at a time: database, then backend, then frontend.

Problem	Possible cause	What to do
`docker: command not found`	Docker is not installed	Repeat Step 1
Port 8080 already in use	Another program uses 8080	Change `"8080:80"` to `"8081:80"` in `compose.yaml`, run `sudo docker compose up -d`, and open http://localhost:8081
Backend keeps restarting	Python error or database not reachable	`sudo docker compose logs backend`
Database shows `unhealthy`	MySQL is still starting or failed	`sudo docker compose logs db`; wait one minute
Page loads but shows an error instead of tasks	Backend or database is down	`sudo docker compose ps` and the backend logs
502 Bad Gateway after rebuilding the backend	Nginx remembers the old backend address	`sudo docker compose restart frontend`
Page edits do not appear	Frontend files are mounted into the container	Reload with Ctrl+Shift+R or run `sudo docker compose restart frontend`
Python code changes do not apply	The backend image must be rebuilt	`sudo docker compose up -d --build backend`
Changed MySQL password has no effect	MySQL read the variables only when the volume was empty	Use the old password, or (**deletes all data**) `sudo docker compose down -v`

17. Clean up

Normal clean-up (keeps your database data)

```
sudo docker compose down
```

Full clean-up (also DELETES the database volume)

```
sudo docker compose down -v
```

> **WARNING:** `down -v` permanently deletes your tasks. Run it only after the instructor has checked your work and you have all your screenshots.

18. Lab submission and review questions

18.1 Submit

- Screenshots 1 to 6 with a one-line caption each.
- Your observations for Experiments 1 to 3.
- Answers to the review questions.

18.2 Review questions

1. What is the difference between a Docker image and a container?
2. Which images in this lab came from Docker Hub and which did you build? Why did the backend need a custom image?

3 What does Nginx do besides serving the web page?

4 Explain `8080:80`. Why do the backend and database have no published ports?

5 How does the backend find the database? (Hint: look at `DB_HOST`.)

6 Why can the frontend container not reach the database?

7 What is the difference between a Docker network and a Docker volume?

8 Why did the tasks survive `docker compose down`? What would have happened with `down -v`?

9 Why are the passwords in `.env` and not in `app.py`?

10 What is the difference between a multi-container application and a multi-node cluster? What would change if each tier ran on its own cloud VM?

Further reading (official documentation)

- Install Docker Engine on Ubuntu: docs.docker.com/engine/install/ubuntu
- Docker Compose: docs.docker.com/compose
- Compose file reference: docs.docker.com/reference/compose-file
- Docker Hub images: hub.docker.com/_/nginx, hub.docker.com/_/python, hub.docker.com/_/mysql

PART 2 - ASSIGNMENT

19. Assignment: Student Course Registration System

Item	Details
Goal	Deploy a **different and more complex** multi-tier application with Docker Compose, using what you learned in the lab
Due date	One week after the lab: _____
Work type	Individual
Submit	Report (PDF or Word) + project folder (ZIP or Git link) + short live demo

19.1 The application

Use a ready-made open-source application with the same features, or write a small one yourself. If you choose another application, get your instructor's approval first. It must provide:

- **Courses:** add, list and delete courses (code, title, number of seats).
- **Students:** add and list students (name, email).
- **Registration:** register a student in a course, cancel a registration, and list the students of each course.
- **Business rules:** a student cannot register twice in the same course; a course cannot accept more students than its seats; emails and course codes must be unique.
- **Database:** at least three related tables (`students`, `courses`, `registrations`) with foreign keys and some sample data.

19.2 Technology

Your stack must be **different from the lab in at least one tier**. Examples:

Tier	Lab used	You may use
Frontend	HTML/JavaScript + Nginx	React, Vue or Angular (built, then served by Nginx), or plain HTML
Backend	Python Flask	Node.js/Express, Python FastAPI or Django, PHP, Java Spring Boot
Database	MySQL 8.4	PostgreSQL, MariaDB, MySQL

19.3 Tasks

Task	What to do	Evidence
1. Environment	Install Docker and Compose, run `hello-world`. Pull the images you will use. Explain image vs container.	Versions, `docker images`
2. Build the application	Create the project: your own backend Dockerfile, frontend and database configuration.	Project tree, Dockerfile, code
3. Deploy with Compose	One `compose.yaml` with three services. Passwords in a `.env` file. A database healthcheck. `restart` policy on all services.	`compose.yaml`, `docker compose ps`
4. Networking	At least two networks so that the frontend cannot reach the database. Only the frontend publishes a port. Nginx forwards `/api/` to the backend. Explain how each tier finds the next one.	`network inspect`, explanation
5. Persistence	Add at least three courses and two students, register students, run `down` then `up`, and show the data is still there. Explain the volume.	Screenshots before and after
6. Monitoring and troubleshooting	Show `ps` and the logs of each service. Stop one tier on purpose, describe the symptoms and how you diagnosed and fixed it.	Logs, short write-up
7. Individual change	Publish the frontend on host port **8000 + the last two digits of your student ID** (for example ID ending in 37 gives port 8037) and show it working.	Screenshot with port visible

19.4 Bonus (up to 10 marks)

- **Admin tool (3 marks):** add Adminer (or phpMyAdmin) as a fourth service, published only on `127.0.0.1` so it is not reachable from other machines.
- **Cache (3 marks):** add Redis on its own container and cache one API response. Show the effect.
- **Cloud (4 marks):** run the same project on one AWS EC2 Ubuntu instance (install Docker, copy the project, open only the needed port in the security group). Provide the public URL and screenshots.

19.5 Deliverables

1. **Architecture diagram** drawn by you (containers, networks, volume, ports).
2. **Project structure** and the full contents of `compose.yaml`, the Dockerfile(s) and the main application code (with passwords removed).
3. **Screenshots** of the running application and of the outputs listed in the Evidence column.
4. **Persistence evidence** (before and after `down` / `up`).
5. **Explanation** of networking, volumes and any troubleshooting you did (about one page).
6. **Live demo** (about 5 minutes): start the project, use the application, show the data in the database, and stop one tier to show what happens.

19.6 Marking scheme

Component	Marks
Docker installation and verification	10
Correct application configuration (Dockerfile, Compose, .env)	20
Successful deployment of all tiers and application features	20
Networking explanation and inspection	15
Database persistence test	15
Monitoring and troubleshooting	10
Report quality and evidence	10
Total	**100**
Bonus	+10

NOTE: Late submission follows the course policy. Identical reports or configurations from different students are treated as plagiarism. Task 7 gives every student a different port, so copied screenshots are easy to spot.